Sequence diagrams

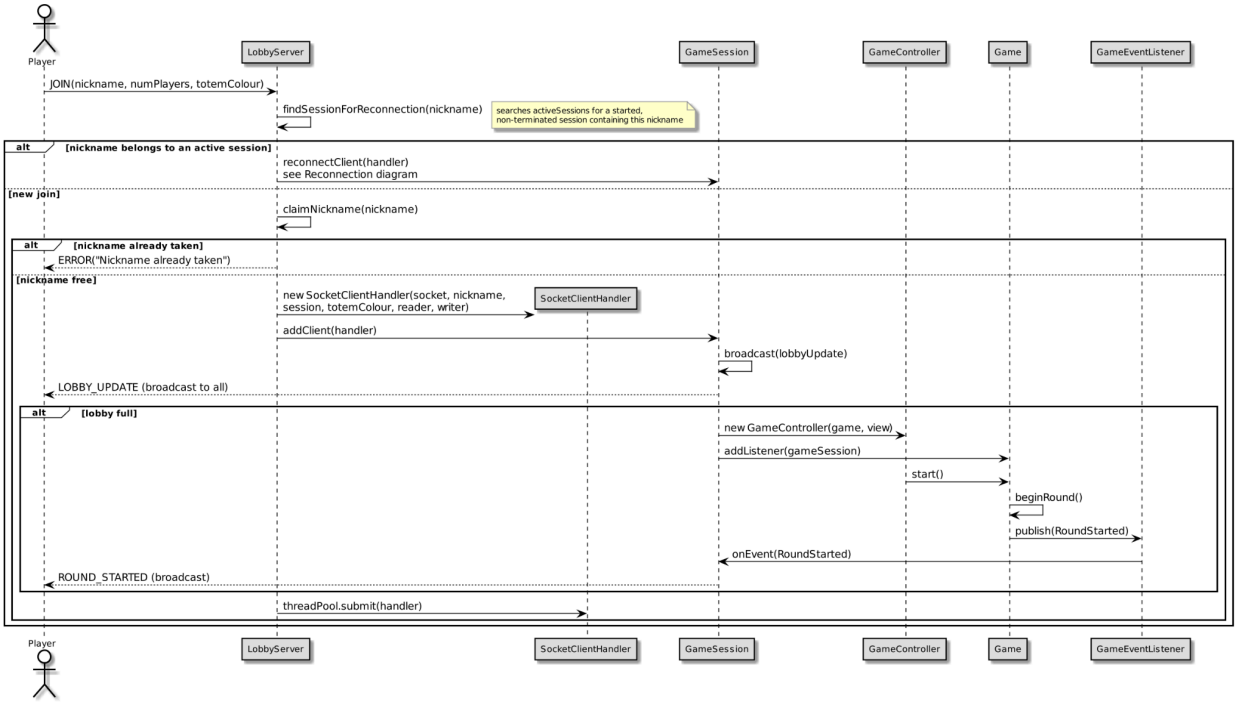
D. Paredes, G. Pastori, D. Prenga, S. Rampini

a.a 2025-2026

Indice dei sequence diagrams:

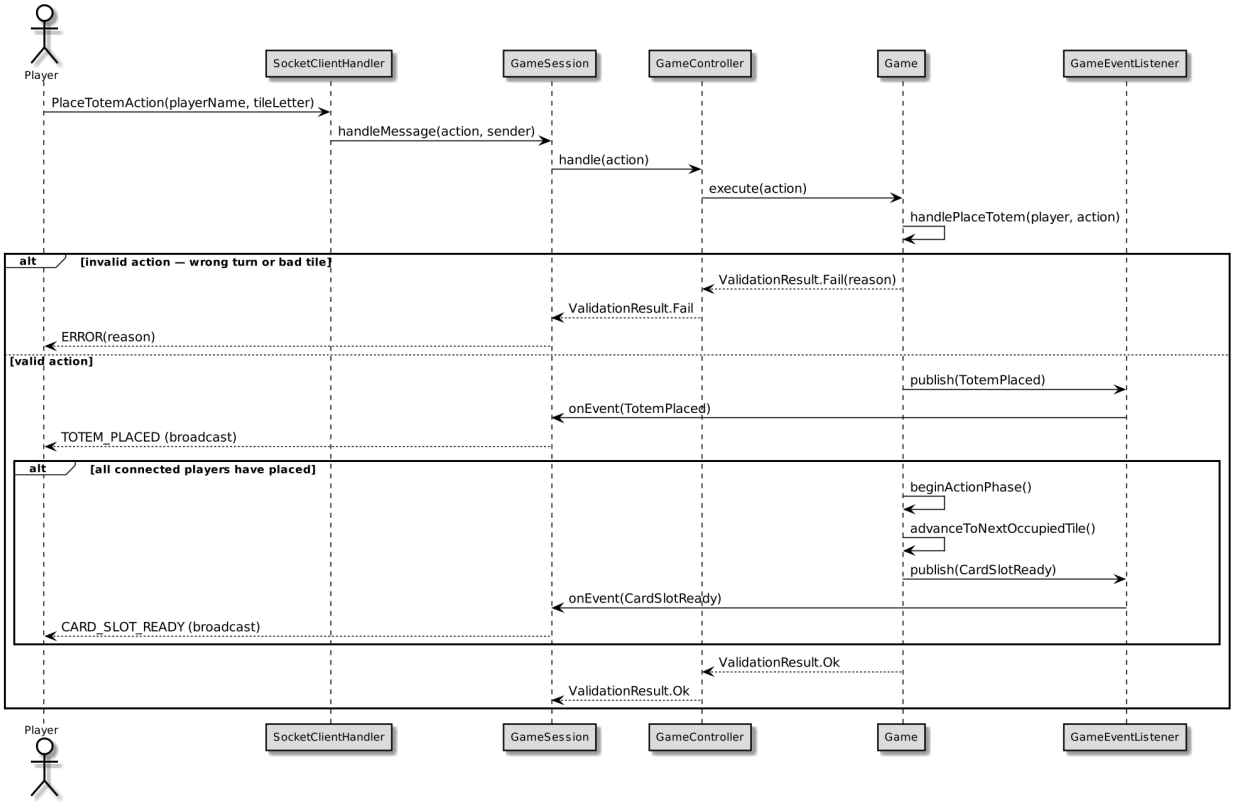
Join game / Lobby ……3
Place totem ……4
Take card ……5
Post round phase ..……6
Reconnection ..……...8

Join Game / Lobby



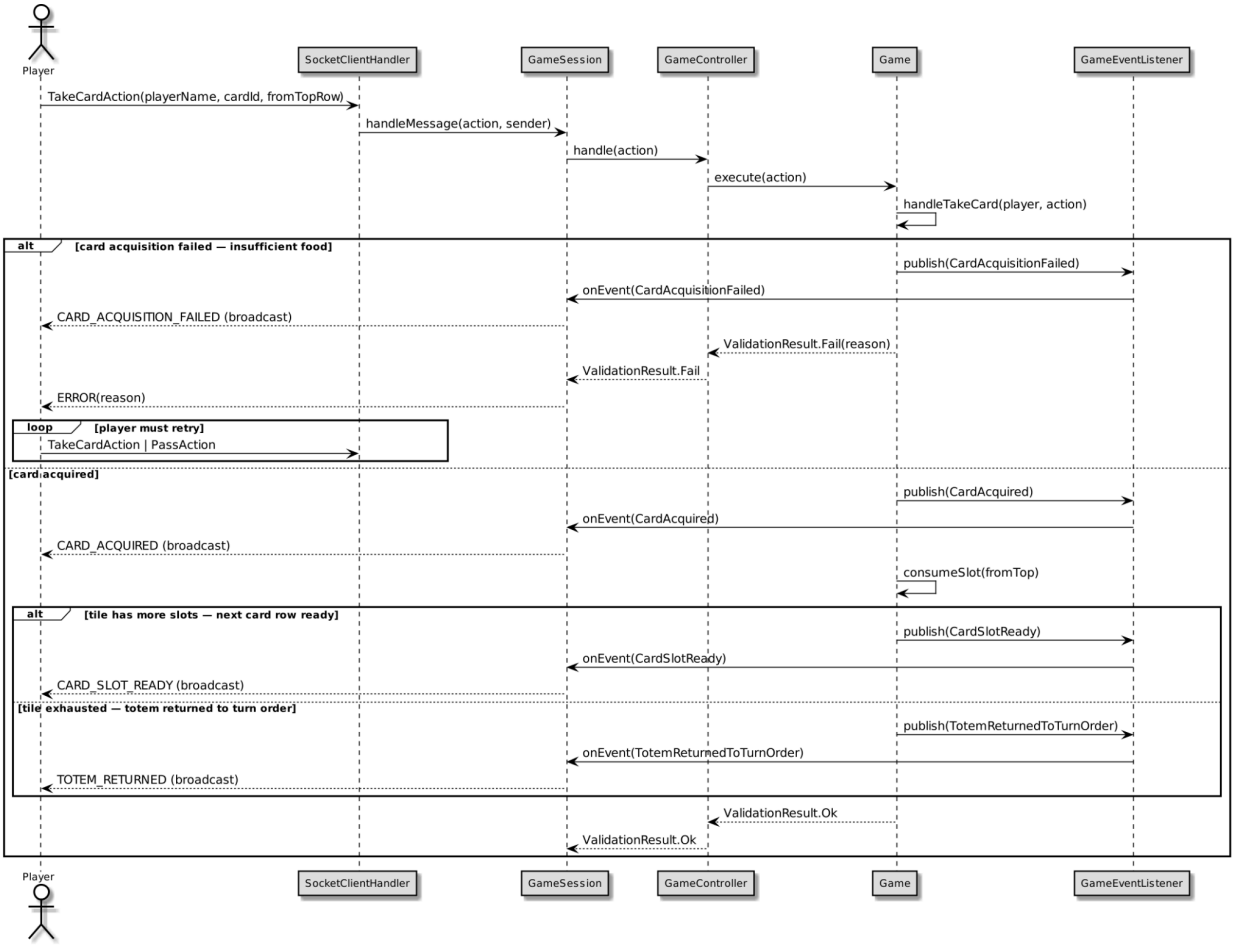
A player joins a game session or reconnects to an existing one. The request is initially handled by the LobbyServer, which first checks whether the provided nickname belongs to an active session. If a reconnection is detected, the request is delegated to the reconnection workflow. Otherwise, the server validates the nickname and creates a new SocketClientHandler for the player. The handler is registered inside the GameSession, which immediately broadcasts an updated lobby state to all connected clients. When the lobby reaches the required number of players, the GameSession creates the GameController, registers itself as a listener, and starts the game. The Game then publishes a RoundStarted event through the observer mechanism. Finally, the GameSession receives the event and broadcasts a ROUND_STARTED message to all participants, marking the beginning of the first round.

Place Totem



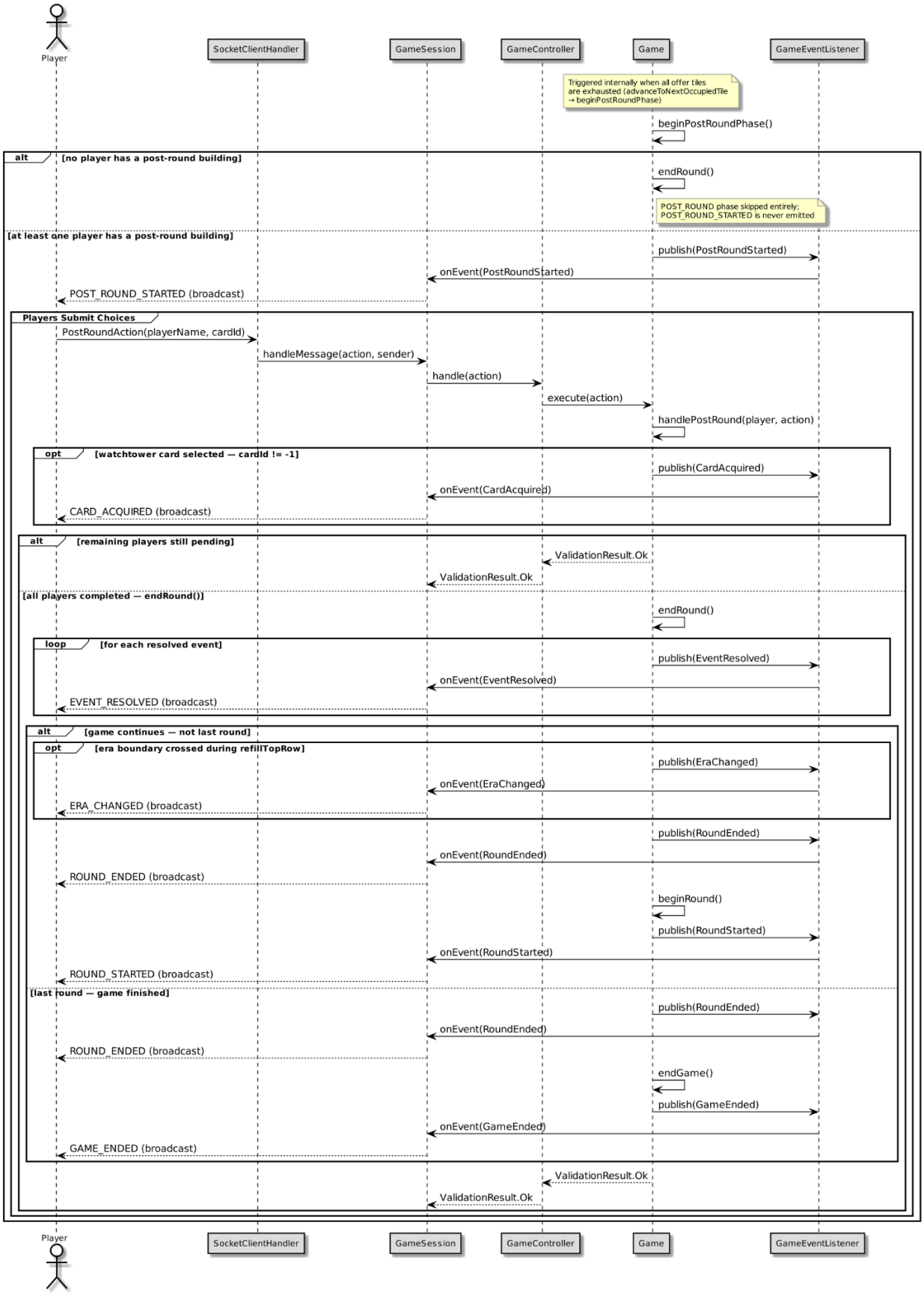
A player places a totem on a board tile. The action is sent by the client and propagated through SocketClientHandler, GameSession, and GameController until it reaches the Game model. The Game validates whether the move is legal, checking turn ownership and tile availability. If the action is invalid, a ValidationResult.Fail is returned and the GameSession sends an ERROR message to the player. If the placement succeeds, the Game publishes a TotemPlaced event that is propagated to the GameSession through the observer pattern. The corresponding TOTEM_PLACED message is then broadcast to all players. When all connected players have completed placement, the Game automatically starts the action phase and publishes a CardSlotReady event. This event is translated into a CARD_SLOT_READY broadcast that allows players to begin selecting cards.

Take card



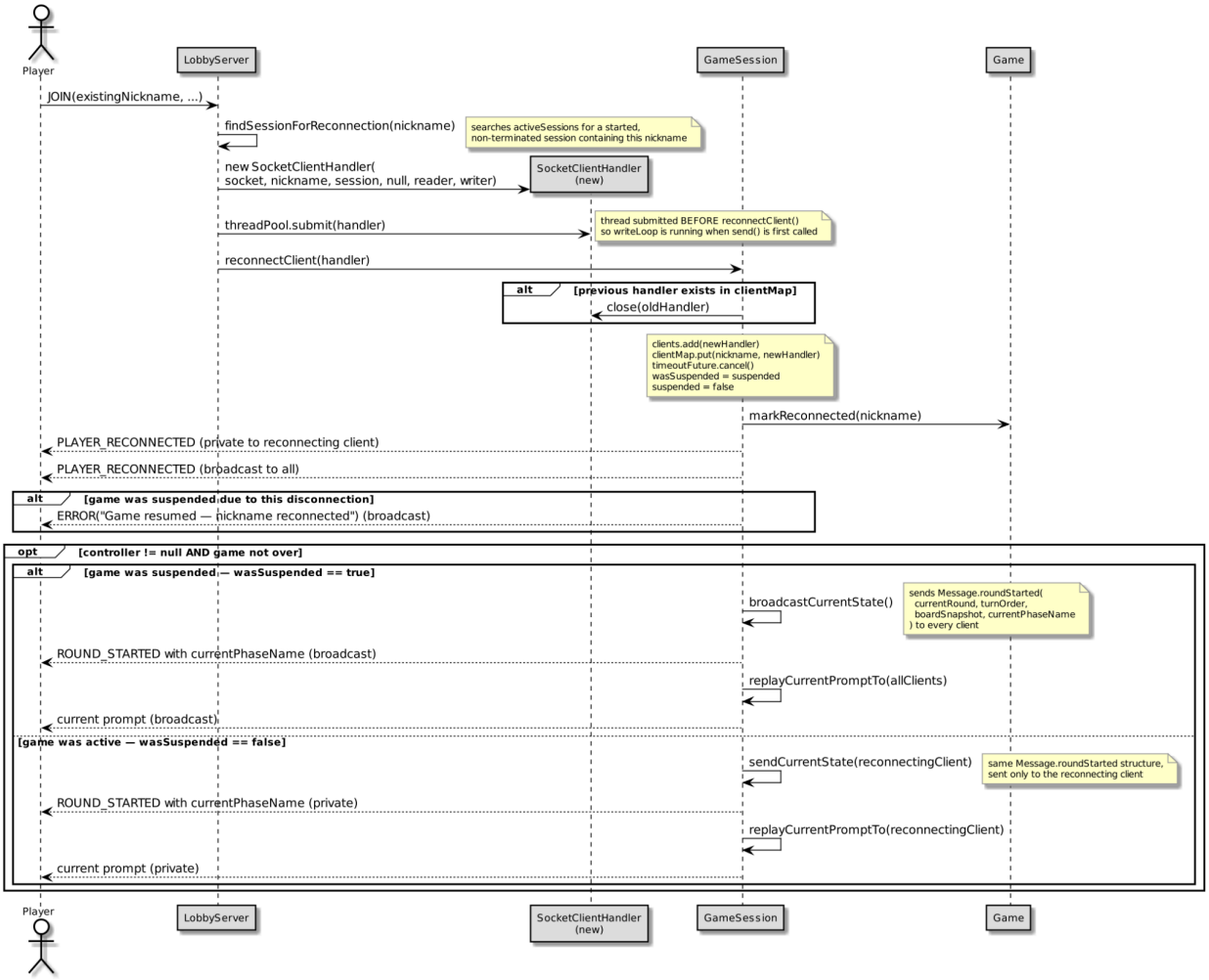
A player sends a TakeCardAction specifying the desired card and its position. The request travels through the network and controller layers before reaching the Game logic. If the player lacks the resources required to acquire the selected card, the Game publishes a CardAcquisitionFailed event and returns a ValidationResult.Fail. The GameSession broadcasts CARD_ACQUISITION_FAILED and also sends an ERROR message to the player. The player may then retry with another card or submit a PassAction. When a card is successfully acquired, the Game publishes a CardAcquired event before completing the action. The occupied card slot is consumed and either a CardSlotReady event or a TotemReturnedToTurnOrder event is generated. These events are converted into CARD_SLOT_READY or TOTEM_RETURNED messages and broadcast to all players. The action finally concludes with a successful ValidationResult.Ok

Post-Round Phase



The post-round phase occurs after all action tiles have been resolved. If at least one player owns a building with a post-round ability, the Game enters the phase and publishes a PostRoundStarted event. The GameSession receives the notification and broadcasts POST_ROUND_STARTED to all participants. Players then submit PostRoundAction messages, optionally selecting a card granted by special buildings such as the Watchtower. Whenever a card is selected, the Game publishes a CardAcquired event that is immediately broadcast to every client. Once all eligible players have completed their choices, the Game executes endRound(), resolving all pending board events and publishing EventResolved notifications. If an era transition occurs, an EraChanged event is also emitted. Finally, the round ends with a RoundEnded event and the game either starts a new round by publishing RoundStarted or terminates by publishing GameEnded.

Reconnection



The system restores a disconnected player during an ongoing match. When a player reconnects using an existing nickname, the LobbyServer searches active sessions for a matching participant. A new SocketClientHandler is created and its communication thread is started before the reconnection procedure begins, ensuring that outgoing messages are not lost. The GameSession replaces the old connection, updates its internal client structures, and marks the player as reconnected inside the Game model. Two PLAYER_RECONNECTED messages are generated: one sent directly to the reconnecting client and another broadcast to all connected players. If the game had been suspended because of the disconnection, an additional notification informs everyone that the match has resumed. The GameSession then synchronizes the current game state and replays the active prompt. Depending on whether the game was suspended or still active, this synchronization is either broadcast to all players or sent only to the reconnecting client.